

1. My name is Sai charan.
2. I ~~am~~ am 21 years old.
3. I ~~am~~Studying in KIET.
4. I ~~am~~ saw a movie last weekend.
5. LLM stands for large language model.
6. SLM stands for small language model.
7. The climate is very cool in My area.
8. The nature is very beautiful.
9. AI stands for Artificial intelligence.
10. Python is a programming language.

Tokenized sentences:-

1. ["my", "name", "is", "sai", "charan"]
2. ["i", "am", "21", "years", "old"]
3. ["I", "am", "studying", "in", "kiet"]
4. ["I", "saw", "a", "movie", "last", "weekend"]
5. ["LLM", "stands", "for", "large",
"language", "model"]
6. ["SLM", "stands", "for", "small",
"language", "model"]

7. "The", "climate", "is", "very", "cool"
"in", "my", "area".

8. ["The", "nature", "is", "very", "beautiful"]

9. ["AI", "stands", "for", "Artificial",
"Intelligence"]

10. ["Python", "is", "a", "programming",
"language"]

Embedding ex

Embedding concept:-

Example 1:-

"I saw a movie last weekend"

tokens:-

["I", "saw", "a", "movie", "last", "weekend"]

sample embedding:-

<u>token</u>	<u>embedding vector</u>
I	[0.1, 0.2, 0.3]
saw	[0.4, 0.5, 0.6]
a	[0.0, 0.1, 0.0]
movie	[0.8, 0.7, 0.9]
last	[0.2, 0.3, 0.1]
weekend	[0.6, 0.5, 0.4]

Example 2:-

"The nature is very beautiful"

Tokens:-

["The", "nature", "is", "very", "beautiful"]

sample embedding:-

<u>Token</u>	<u>embedding vector</u>
The	$[0.91, 0.16, 0.49]$
nature	$[0.87, 0.19, 0.51]$
is	$[0.86, 0.17, 0.47]$
very	$[0.22, 0.53, 0.15]$
beautiful	$[0.93, 0.12, 0.33]$

Analysis of Sentences using RNN and transformed approaches.

sentence 1:

"my name is sai charan".

Tokens:-

{ "my", "name", "is", "sai", "charan" }

RNN Analysis:-

- The RNN processes the sentence on token at a time.
- It first reads "my" the updates its hidden state.
- Next, it reads "name", using information from "my".
- It continues with "is", remembering previous words.

- Then it processes "sai", combining it with past context.
- Finally, It reads "charan", which contains full text of the sentences.

Processing order:

my $\rightarrow$ name $\rightarrow$ is $\rightarrow$ sai $\rightarrow$ charan.

Transformer Analysis:

- The transformer processes the entire sentence at once.
- All words like "my, name, is, sai, charan" are given to the model together.
- Each word looks at all other words using self-attention.

• "Sai" can directly attend to "my" and "name" without waiting step by step.

• "Charan" also gets full context from all previous words simultaneously.

Processing:-

$[my] \leftrightarrow [name] \leftrightarrow [is] \leftrightarrow [sai] \leftrightarrow [charan]$

(All words interact with each other at the same time).

Sentence 2:-

"Python is a programming language".

Tokens:-

"python", "is", "a", "programming",
"language".

RNN Analysis:-

- The RNN processes the sentences one word at a time.
- The first reads "python" and updates its hidden state.
- Next, It reads "is", using information from "python".
- Then it process "a", remembering previous context.
- After that, It reads "programming" combining it with earlier words.

- Finally, It reads "language", which contains the full meaning of the sentences.

- The final hidden state represents the entire sentence meaning.

Processing order:

python $\rightarrow$ is $\rightarrow$ a $\rightarrow$ programming $\rightarrow$ language.

Transformer Analysis:-

- The transformer processes all five tokens in parallel.

- every token can attend to every other token using self-attention.

- "programming" directly connects with "language" and "python".
- "language" understands full context by looking at all words simultaneously.
- Finally, all contextual information is combined into a sentence embedding.

Processing:

[python] $\leftrightarrow$ [is] $\leftrightarrow$ [a] $\leftrightarrow$ [programming]
 $\leftrightarrow$ [language]

Comparison of outputs RNN and Transformer model.

Sentence

"My name is Sai charan".

RNN output:

- Reads sentence word by word and builds memory sequentially.
- Identifies "Sai charan" as the name after processing all words.
- Depends on hidden state and works well for short sentences.

Transformer output:

- Reads all words at the same time using self-attention.
- Directly connects "Sai" and "charan" to form meaning.
- Gives better and faster contextual understanding.

Sentence 2:-

"Python is a programming language."

RNN output:-

- Reads the sentences word by word in sequential order.
- Builds meaning using hidden state memory. step by step.
- Final output depends on last hidden state.

Transformed output:-

- Processes all words at the same time using self-attention.
- Connects "Python" with "programming language" directly.
- Produces better contextual understanding of the sentences.

Sentence 2:-

"Python is a programming language."

RNN output:-

- Reads the sentences word by word in sequential order.
- Builds meaning using hidden state memory. step by step.
- Final output depends on last hidden state.

Transformed output:-

- Processes all words at the same time using self-attention.
- Connects "Python" with "programming language" directly.
- Produces better contextual understanding of the sentences.

Accuracy.

RNN : sequential (one word at a time)

Transformed:-

Parallel (all words together)

Context understanding:-

RNN:- Good for short sentences
like "Python is a programming
language".

Transformer:-

Excellent for both short and
long sentences with better con-
text understanding.

Feature	RNN	Transformers
Scalability	Less efficient for long text	Highly efficient for long text.
Overall accuracy.	Good	Better than RNN.